Using MsPASS for Data Processing on HPC and Cloud Systems

Session 2: Parallel Processing

Prof. Gary L. Pavlis
Department of Earth and Atmospheric Sciences
Indiana University, Bloomington, IN





Agenda Today

- Questions from Homework
- Lecture1: Parallel Processing
  - Review of key points from precourse lecture on this topic
  - MsPASS services
  - Parallel containers
  - Serial database driven workflows in MsPASS
  - Converting serial workflow to a parallel one
  - Futures
- Hands on work on GeoLab
  - Simple serial process prototyping
  - Debugging serial prototype
  - Conversion to parallel
  - Monitoring performance with dask diagnostics
  - Example of more complicated workflow and conversion to parallel
- Lecture 2: Issues with stock dask scheduler
  - Oversubscribing CPUs
  - Memory management
  - Solution in newest MsPASS release using Futures and a “sliding window”

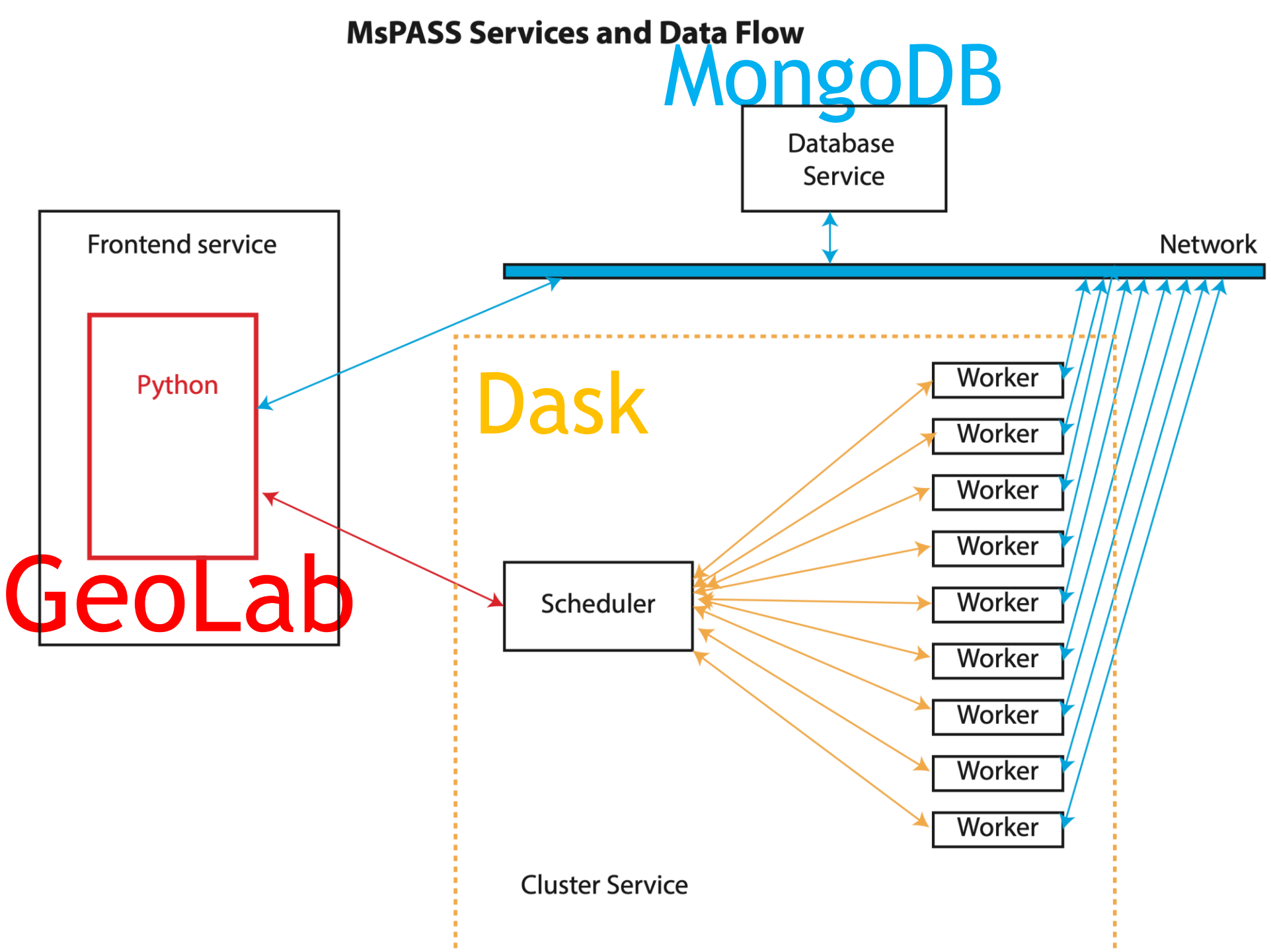
Key points from Precourse Lecture

- HPC and Cloud computers are clusters
  - Core – cpu
  - Blade/Node – one standalone computer
  - Shared versus distributed memory
- Parallel processing concepts
  - Threads – shared memory only
  - MPI – standard framework for simulation code
  - Foreman/worker – standard big data approach (what MsPASS uses)

With that background a short poll to help me understand my audience

Service Concept

- Generic abstraction of UNIX daemon (background process)
- Central to cloud computing
- Main ideas
  - A service is a piece of software that runs in the background
  - A service responds to requests via an API
  - A service is often conceptualized as a black box with a name tag


MsPASS Services and Data Flow
MongoDB
Database Service
Network
Frontend service
Python
GeoLab
Dask
Scheduler
Worker
Worker
Worker
Worker
Worker
Worker
Worker
Worker
Cluster Service

You interact with services via an API (Application Programming Interface)

- Database Service: pymongo
- Cluster service: dask distributed
- Frontend service: interactive with jupyter notebook or batch python script run

Why do I need to know this?

- You don’t to get started – Monday was a case in point
- When this material is important
  - When you are tuning a workflow for performance
  - When you are configuring an HPC cluster (Friday topic)
  - When something goes wrong – it is possible to crash any of these services

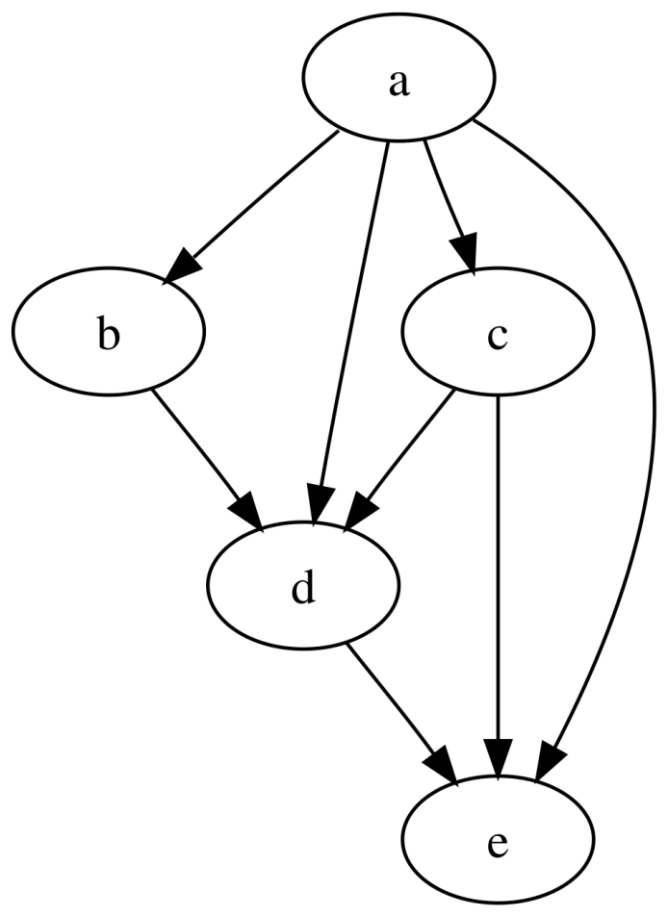
Dask distributed

- Generic parallel processing framework of MsPASS
- Importance: provide a generic mechanism to run task in parallel
- Concepts: next slide

Generic Parallel Processing

- Recall Precourse lecture concepts
  - Baking a cake analogy
  - Foreman-worker model
  - Directed Acyclic Graph – under the hood detail
- Foreman == scheduler
- Workers do what the boss tells them to do
- DAG and “futures” (dask) support generic parallel algorithm

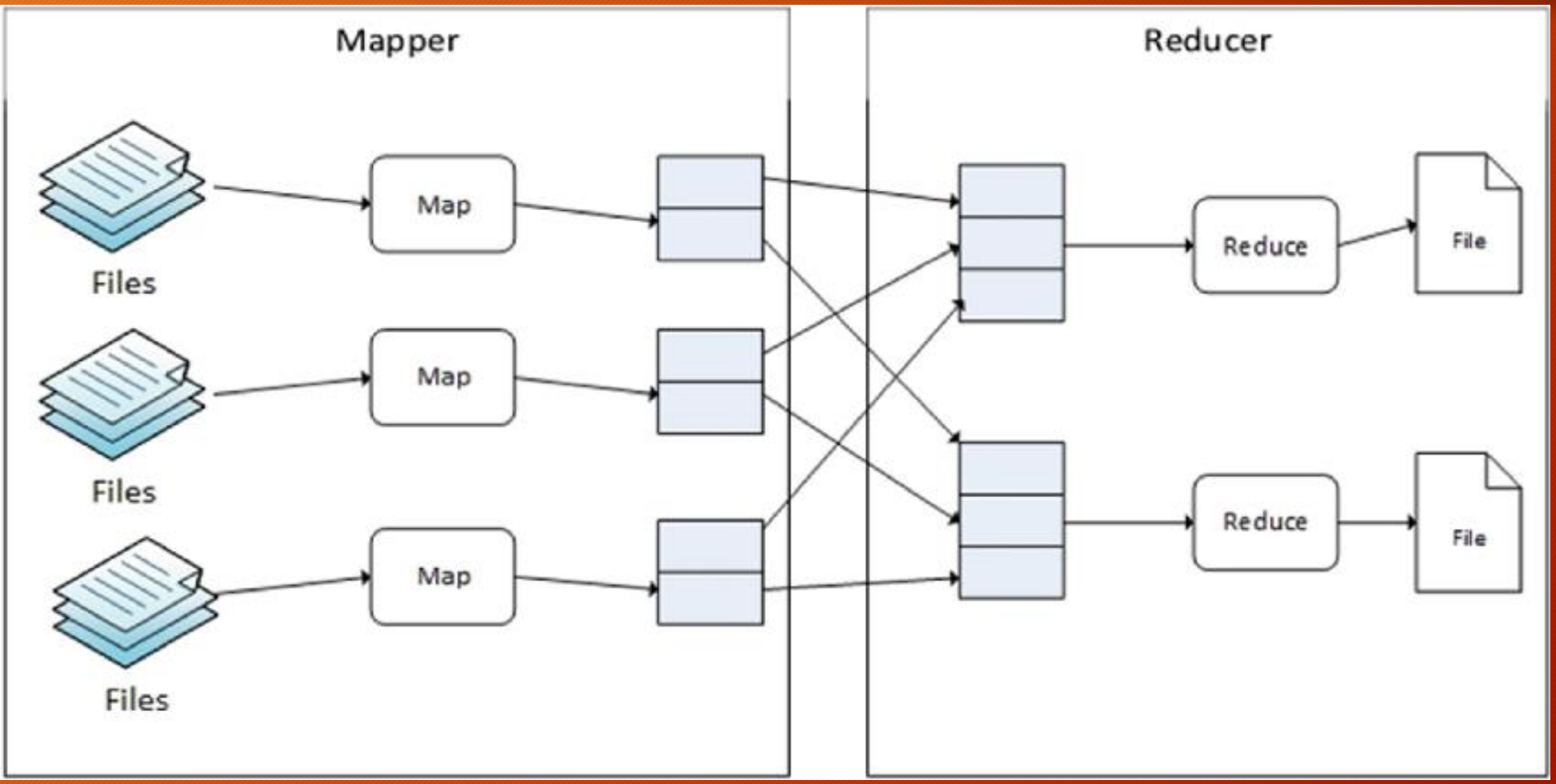
Directed Acyclic Graph – how schedulers manage data flow



- Boxes a, b, c, d, and e are compute tasks
- Arrows show data flow
- “Acyclic” – data flows only one way
- Key point is this is a generic scheduling method
  - Good for goal of flexibility in research computing
  - Can be bad for efficiency (performance)

From: https://en.wikipedia.org/wiki/Directed_acyclic_graph

Generic map-reduce concept

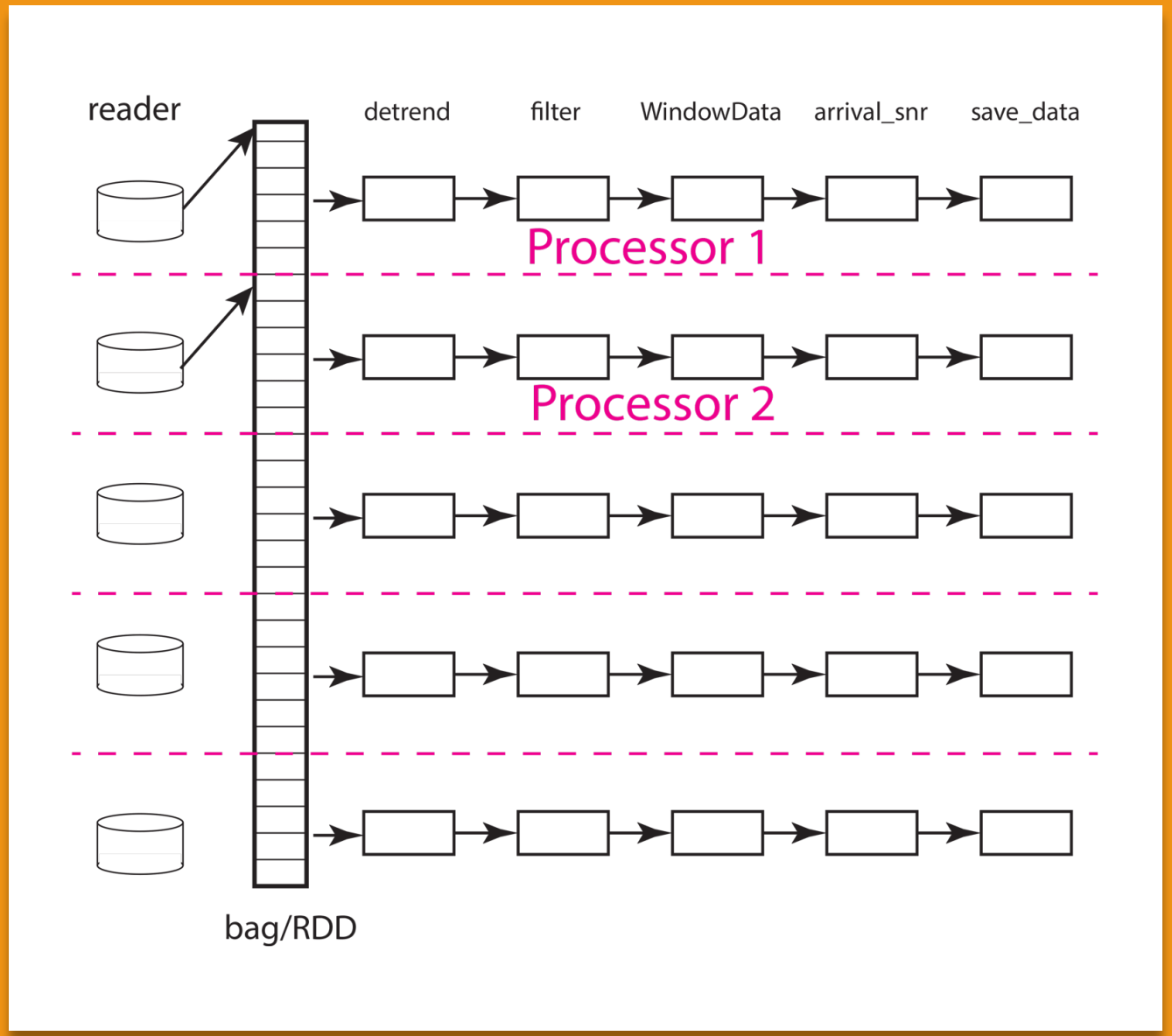


Applications:

- Data input is a (huge) list of the same things
- Map – apply a common algorithm to each thing
- Reduce
  - Combine things
  - Order cannot matter

Most seismic processing workflows are pipelines

- Origin (reader) is a read operator that loads data into a parallel container (bad/rdd)
- Process each datum with 5 processing functions (boxes with names at top)
- Example shows how scheduler might pass work to 2 processors running in parallel
- A pure pipeline DAG is the definition of “embarrassingly parallel”



Some key concepts

- Parallel container (bag) – notebooks will make this clearer
- Workers run processing functions (detrend, filter, WindowData, arrival_snr, and save_data in previous animated figure)
- Scheduler tells workers which components of the data set (things in bag/RDD container) to run when
  - Animation show that happening sequentially
  - In real life the order is not sequential
  - Scheduler has to guess how many data to handle simultaneously from
    - Memory use
    - How busy CPUs are
  - We will see a real time version running session2 notebook

Algorithm restrictions of map-reduce

- All algorithms must be expressed as a function
- Function input as arg0 should always be the same
- Arguments must be constant for all data
- Return type can and often is different from arg0 input
- Should never throw an exception unless you want the workflow to abort for the entire dataset
- Reduce has additional restrictions
- Use wrapper functions to adapt existing code

See this section of the MsPASS User Manual for summary of MsPASS algorithms

Let’s look briefly at a dask example

- Open notebook: ParallelProcessingFundamentals.ipynb”
  - Quick look at an example pipeline workflow
- Open notebook “SeismicReduceOperatorExample.ipynb”
  - Quick look to show you flow structure
  - Run and review this notebook at your leisure – it is largely a reference
  - An important point is to date we’ve not found a useful example of a Reduce operator in seismic processing – see closing text box of the example

Serial Prototyping

- ALWAYS the starting point for any parallel workflow
  - Get all the syntax right
  - Assure the suite of algorithms work together
- If you don’t need parallel processing don’t bother (feasibility)
- Used to establish timing data to estimate resources for full data set

Parallelization example:

Serial Form

```
cursor=db.wfcollection.find(query)
for doc in cursor:
  d = db.read_data(doc)
  d = proc1(d)
  d = proc2(d)
  db.save_data(d)
```

Parallel Version (dask)

```
cursor = db.wfcollection.find(query)
df = read_to_dataframe(db,cursor)
mybag = read_distributed_data(df,db)
mybag = mybag.map(proc1)
mybag = mybag.map(proc2)
out = write_distributed_data(mybag,db)
```

Creation

Process

Save result

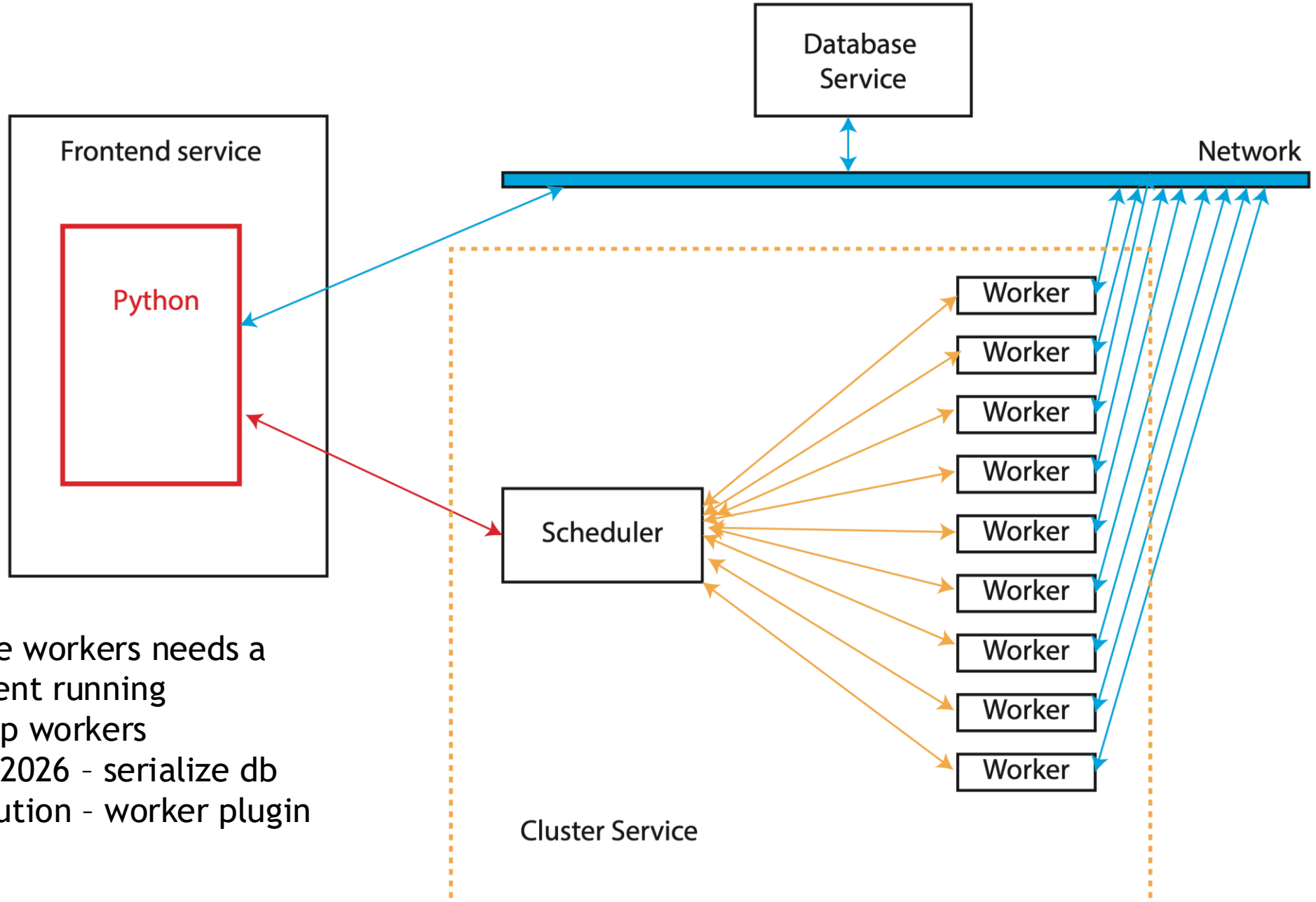
Note:

1. Above omits optional arguments for clarity that are nearly always required
2. We'll look at real examples shortly in notebooks

An important detail: parallel clients

- Problem happens when workers need access the MongoDB
- All MsPASS seismic read and write functions require MongoDB
- Recall MsPASS services wiring diagram

MsPASS Services and Data Flow



- Each of those workers needs a database client running
- How to set up workers
  - Prior to 2026 – serialize db
  - New solution – worker plugin

How to set up worker clients:

```
from mspasspy.util.db_utils import MongoDBWorker
from mspasspy.client import Client

mspass_client = Client()
dask_client = mspass_client.get_scheduler()
parallel_dbworker = MongoDBWorker(mspass_client)
dask_client.register_worker_plugin(parallel_dbworker)
```

IMPORTANT WARNING:

- Prior to earlier this year we used a serialization approach to push database handle to workers
- That works for small data sets, but fails with very large data sets
- We have NOT yet fixed all the documentation
  - Most of the user manual example have the old (wrong) form
  - Most of our tutorials are wrong (Earthscope2026 is the main exception)
  - All previous MsPASS Earthscope tutorials used the old serialization approach
- Use the tutorials from this course or follow guidance in this page of the MsPASS User Manual

To GeoLab

- We'll first go through "ParallelProcessingFundamentals.ipynb"
  - You should already have that open
- Step 2 is running the "Session2.ipynb" notebook

Part 2: Dask Futures

- Distributed memory implementation of concurrent.futures that became a standard in python 3.2 (2011)
- The standard python futures interface is a useful thing to know independent of MsPASS (numerous online sources)
- Dask uses the futures API under the hood for almost everything
- More primitive so it has fewer safeties

Dask Futures API key components

- submit – Send a function to run one datum on the cluster
- future.result – block until a single future completes
- gather – block while a list of futures complete and return results as a list
- as_completed – handle a list of futures as they finish
- scatter – broadcast data to workers (improves performance for a large data set needing common large objects like a travel time calbulator)

Converting a pipeline workflow for futures

Map Operator Form

```
Import dask.bag as dbg
mybag = dbg.from_sequence(x)
mybag = mybag.map(convert_to_query)
mybag = mybag.map(read_data)
mybag = mybag.map(proc1)
mybag = mybag.map(proc2)
mybag = mybag.map(db.save_data)
mybag.compute()
```

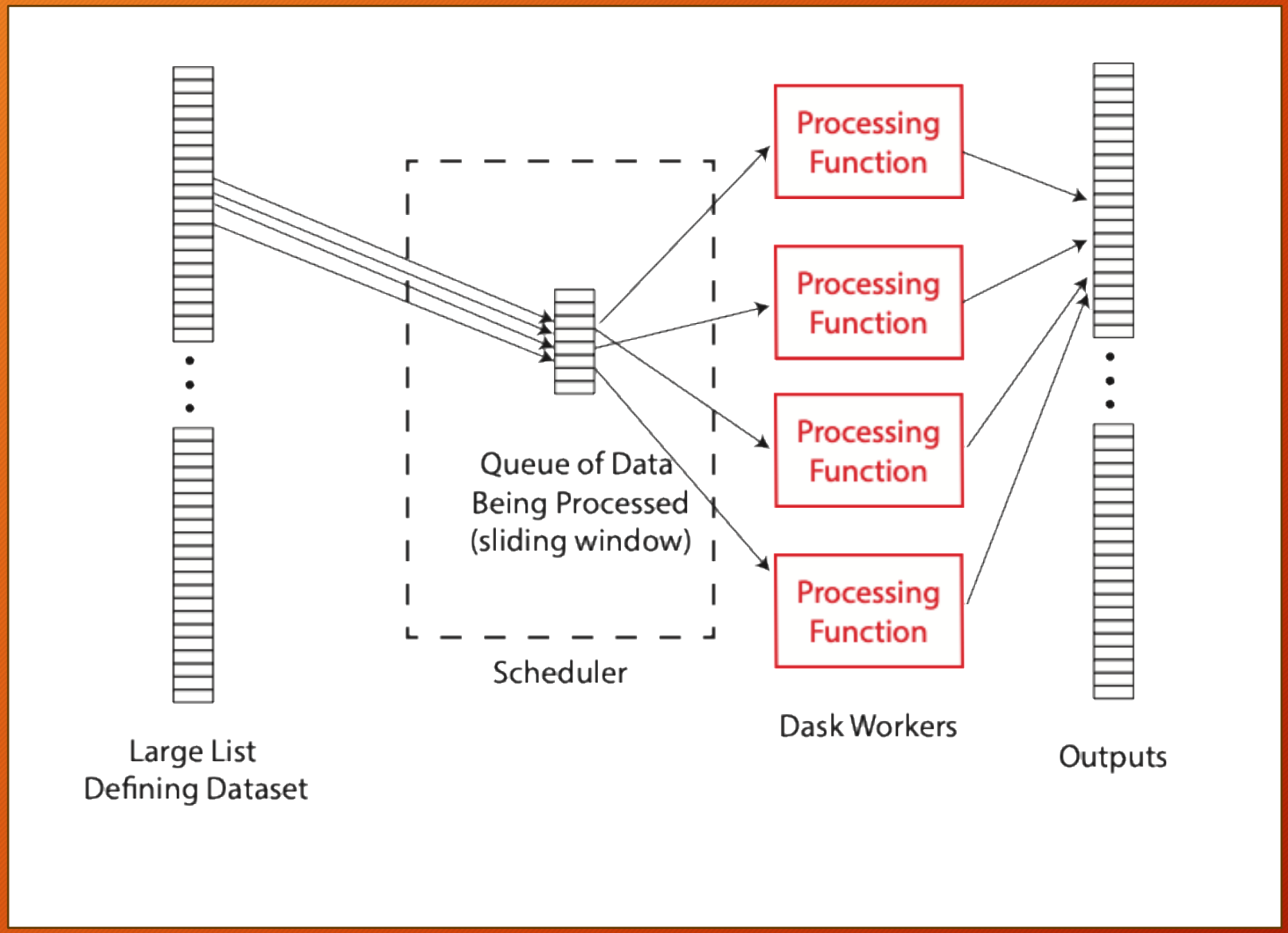
Futures form

```
dask_client = mspass_client.get_scheduler()
def process_one(x):
    q = convert_to_query(x)
    d = read_data(q)
    d = proc1(d)
    d = proc2(d)
    return d
data_list = generate_list_to_drive_processing()
Flist=list()
for x in data_list:
    f = dask_client.submit(process_one,x)
    Flist.append(f)
Result = dask_clent.gather(Flist)
```

Problems with that crude example

- If the list is huge it will overwhelm the scheduler
- Memory use will grow out of control
- Return result can be huge

Solution: MsPASS sliding_window_pipeline



- Data submitted sequentially to queue
- Scheduler decides which worker gets which datum to handle
- Processing function takes input form list and emits something
- Output is a list with length the same as input
- Not illustrated is that when a reader or writer blocks the scheduler gives it more work (more data being handled simultaneously than number of workers)

Advantages of Sliding Window Algorithm

- Memory used is predictable (queue size time maximum memory required by function during processing)
- Let’s the operating system manage resources more effectively
- Proven capability to handle millions of inputs simultaneously IF the input is list of documents, ids, or database queries